# Fondamenti di elaborazione dei segnali

N. 9 Sistemi e filtri digitali

# Sommario

- Estensione delle proprietà dei segnali al caso discreto
- Sistemi numerici e convoluzione discreta
- Filtri discreti: FIR E IIR
  - Esempi di filtri FIR LPF (media mobile, gaussiano)
  - Esempi di filtri FIR HPF (gradiente, laplaciana)
  - Filtri iterativi
  - Filtri IIR tramite autoregressione
- Cenno ai filtri non lineari
  - Rimappatura livelli
  - Filtri di rango: max, min e mediano

# Proprietà dei segnali discreti

- Possiamo estendere, talvolta in modo immediato altre volte meno, le proprietà che avevamo visto per i segnali analogici ai segnali digitali
  - Richiamiamo qui le principali proprietà che avevamo visto a inizio corso e vediamo come si applicano nel caso discreto
- Simmetria
  - PARI: $x(t) = x(-t) \ \forall t \ \rightarrow \ x(n) = x(-n) \ \forall n$
  - DISPARI: $x(t) = -x(-t) \ \forall t \ \rightarrow \ x(n) = -x(-n) \ \forall n$
- Guadagno
  - POSITIVO (amplificazione): $\alpha x(t) : \alpha > 1 \ \rightarrow \alpha x(n) : \alpha > 1$
  - NEGATIVO (attenuazione): $\alpha x(t) : \alpha < 1 \ \rightarrow \alpha x(n) : \alpha < 1$

# Proprietà dei segnali discreti

- Fattore di scala: nel dominio digitale, compressione ed espansione corrispondono a **ricampionamenti** del segnale
  - COMPRESSIONE (sottocampionamento): viene realizzata applicando un filtro passa-basso e poi decimando i campioni, il filtro è necessario per non introdurre aliasing
  - ESPANSIONE (sovracampionamento): viene realizzata interpolando nuovi campioni nella sequenza o replicandoli e poi filtrando sempre con un passa-basso

*NB: il segnale di uscita deve essere sempre una sequenza discreta di campioni equispaziati, quindi se il fattore di scala non è intero occorre interpolare anche i valori originali*

# Proprietà dei segnali discreti

- Integrazione e derivazione
  - INTEGRAZIONE: diventa somma cumulativa

$$y(t) = \int_{-\infty}^{t} x(\xi)d\xi \rightarrow y(n) = \sum_{k=-\infty}^{n} x(k)$$

  - DERIVAZIONE: diventa differenza finita

$$y(t) = \frac{d}{dt}x(t) \rightarrow y(n) = \frac{1}{T_c}[x(n) - x(n-1)]$$

*NB. Data una sequenza discreta, $T_c$ potrebbe non essere noto, in questo caso calcolo solo la differenza tra i campioni (pongo $T_c$=1)*

# Proprietà dei segnali discreti

- Vediamo l'estensione delle misure statistiche
  - VALOR MEDIO

$$\bar{x} = \lim_{T\to\infty} \frac{1}{T} \int_{-T/2}^{T/2} x(t)dt \longrightarrow \bar{x} = \lim_{K\to\infty} \frac{1}{K} \sum_{k=-\frac{K}{2}}^{\frac{K}{2}} x(k)$$

  - VARIANZA

$$\sigma_x^2 = \overline{x^2} - \bar{x}^2 = \lim_{T\to\infty} \frac{1}{T} \int_{-\frac{T}{2}}^{\frac{T}{2}} (x(t) - \bar{x})^2 dt \longrightarrow \sigma_x^2 = \lim_{K\to\infty} \frac{1}{K} \sum_{k=-\frac{K}{2}}^{\frac{K}{2}} [x(k) - \bar{x}]^2$$

# Proprietà dei segnali discreti

- CORRELAZIONE

$$\mathcal{R}_{xy}(\tau) = \int_{-\infty}^{+\infty} x(t)y(t+\tau)\,\mathrm{d}t \;\longrightarrow \mathcal{R}_{xy}(k) = \sum_{n=-\infty}^{+\infty} x(n)y(n+k)$$

- AUTOCORRELAZIONE

$$\mathcal{R}_{x}(\tau) = \int_{-\infty}^{+\infty} x(t)x(t+\tau)\,\mathrm{d}t \;\longrightarrow \mathcal{R}_{x}(k) = \sum_{n=-\infty}^{+\infty} x(n)x(n+k)$$

# Proprietà dei segnali discreti

- ENERGIA (segnali discreti di energia)

$$E_x = \int_{-\infty}^{+\infty} x^2(t)dt \longrightarrow E_x = \sum_{k=-\infty}^{+\infty} x^2(k)$$

- POTENZA MEDIA (segnali discreti di potenza)

$$P_x = \lim_{T\to\infty} \frac{1}{T} \int_{-T/2}^{+T/2} x^2(t)dt \longrightarrow P_x = \lim_{K\to\infty} \frac{1}{K} \sum_{k=-\frac{K}{2}}^{\frac{K}{2}} x^2(k)$$

# Istogramma

- Dato un segnale discreto, è possibile calcolare la densità di probabilità dei livelli assunti dal segnale
  - In realtà, se dispongo di un numero finito di campioni *N*, le probabilità dei livelli saranno stimate tramite le frequenze relative di occorrenza dei valori
- Si ottiene il cosiddetto **istogramma** del segnale, h(l)

$$h(l) = \frac{1}{N} \sum_{n=1}^{N} \begin{cases} 1 \; se \; x(n) = l \\ 0 \; altrimenti \end{cases} \quad l = 1, \dots, L$$

- NB. L'istogramma è una ddp discreta, quindi gode delle stesse proprietà. In particolare, la somma delle righe (integrale discreto) è pari a 1

# Calcolo istogramma: pseudo-codice

```
int x[N]
float h[K]
load x
for n in 1...N
      h[x[n]] += 1.0
for k in 1...K
      h[k] /= N
return h
```

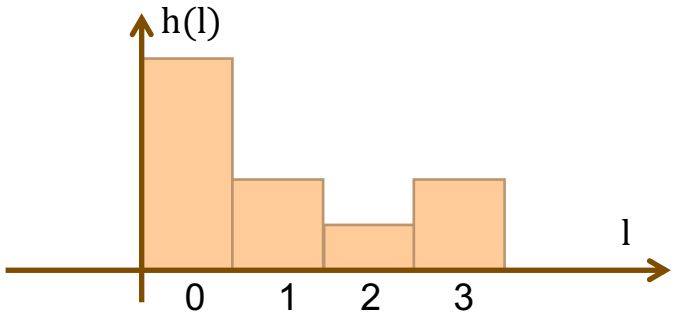
# Istogramma: esempio

- Prendiamo una sequenza discreta di lunghezza 30, con campioni a 2 bit (4 livelli)

  x(n)=[0 0 0 0 1 1 0 0 0 1 3 3 3 0 0 0 1 1 1 0 0 0 0 0 2 3 3 3 2 2]

  - L'istogramma sarà formato da 4 righe (pari al numero di livelli)
  - L'altezza di ogni riga sarà pari al numero di campioni che assumono il livello corrispondente, diviso per il numero di campioni totali 30

  h(0) = 15/30; h(1) = 6/30; h(2)=3/30; h(3)=6/30



# Calcolo statistiche dall'istogramma

- Una volta calcolato l'istogramma è possibile calcolare direttamente altri valori statistici, usando l'istogramma come funzione pdf e la definizione di media statistica
  - Valor medio

$$\bar{x} = \sum_{l=1}^{L} l \cdot h(l)$$

  - Varianza

$$\sigma_x^2 = \sum_{l=1}^{L} (l - \bar{x})^2 \cdot h(l)$$

# Esempio

- Data la sequenza dell'esempio precedente e il relativo istogramma avremo:

$x$(n)=[0 0 0 0 1 1 0 0 0 1 3 3 3 0 0 0 1 1 1 0 0 0 0 0 2 3 3 3 2 2]

$$h(\mathrm{l}) = \frac{1}{30}[15\ 6\ 3\ 6]$$

$$\bar{x} = \frac{1}{30}(0 \cdot 15 + 1 \cdot 6 + 2 \cdot 3 + 3 \cdot 6) = \frac{30}{30} = 1$$

$$\sigma_x^2 = \frac{1}{30}[(0-1)^2 \cdot 15 + (1-1)^2 \cdot 6 + (2-1)^2 \cdot 3 + (3-1)^2 \cdot 6)] = \frac{42}{30} = 1.4$$

*NB. Tipicamente K<<N per cui occorrono molte meno operazioni rispetto al calcolo nel tempo*

# Sistemi numerici

- L'elaborazione di segnali digitali prevede l'utilizzo di sistemi che a loro volta siano digitali (numerici)
  - Un sistema numerico è un sistema che riceve in ingresso un segnale digitale e restituisce un altro segnale digitale
  - Il segnale di uscita può avere caratteristiche uguali o diverse da quelle del segnale di ingresso (dimensionalità, rappresentazione dei campioni)
  - Il sistema realizzerà una qualche **elaborazione sul segnale**, che dipenderà dalle caratteristiche del sistema e da eventuali parametri
  - Come per i sistemi analogici, anche in questo caso potremo avere **sistemi LTI e non-LTI**
  - A differenza dei sistemi analogici, i sistemi digitali sono tipicamente implementati tramite **algoritmi (software)**

# Sistemi numerici LTI

- Partiamo dal caso più semplice, ovvero dalla realizzazione di sistemi numerici di tipo LTI
  - Cominciamo con l'estendere al caso digitale le condizioni viste per sistemi analogici (sovrapposizione effetti e tempo invarianza)
  - LINEARITA' (sovrapposizione degli effetti)

$$\left.\begin{array}{l} f\big(x_1(n)\big) = y_1(n) \\ f\big(x_2(n)\big) = y_2(n) \end{array}\right\} \Longrightarrow f\left(\alpha x_1(n) + \beta x_2(n)\right) = \alpha y_1(n) + \beta\, y_2(n)$$

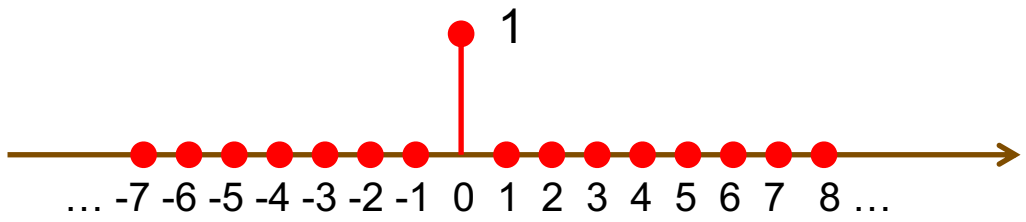
  - TEMPO INVARIANZA

$$f\big(x(n)\big) = y(n) \Longrightarrow f\big(x(n-k)\big) = y(n-k)$$

# Impuso di Kroneker

- Per sistemi LTI, il teorema di convoluzione continua ad essere valido anche nel discreto, tuttavia…
  - prima dobbiamo capire cos'è in questo caso la risposta impulsiva…
  - e prima ancora dobbiamo definire cos'è un impulso nel discreto
  - Partiamo da qui: l'estensione discreta della delta di Dirac è detta impulso di Kroneker, ed è il seguente semplice segnale discreto:

$$\delta(n) = \begin{cases} 1 & se \quad n = 0 \\ 0 & altrove \end{cases}$$



# Proprietà dell'impulso di Kroneker

- L'impulso così definito mantiene tutte le proprietà della delta di Dirac, e in particolare:
  - Il suo integrale (ovvero la sommatoria, essendo nel discreto) vale 1
  - E' una funzione simmetrica → $\delta(n) = \delta(-n)$
  - Un impulso in posizione k, $\delta(n-k)$, moltiplicato per un segnale discreto x(n), estrae il valore del segnale in posizione k, x(n-k)
- A differenza della delta di Dirac, tuttavia, l'impulso di Kroneker non presenta alcun problema matematico
  - E' una funzione discreta perfettamente definita in ogni punto

# Risposta all'impulso discreta

- Definito l'impulso discreto, dobbiamo definire la risposta impulsiva discreta
  - Imponendo in ingresso a un sistema LTI discreto un impulso di Kroneker $\delta(n)$, otterremo in uscita la risposta impulsiva discreta h(n)




  - In generale, l'effetto sarà introdurre un 'allargamento' dell'impulso su una certa sequenza di campioni: per questo viene anche chiamato **Point Spread Function** (PSF, funzione di allargamento del punto)

# Convoluzione discreta

- Definita la risposta impulsiva discreta, rimane da estendere al dominio discreto il concetto di convoluzione
  - Ricordiamo la formula nel continuo:

$$y(t) = x(t) * h(t) = \int_{-\infty}^{+\infty} x(\tau)h(t-\tau)d\tau$$

  - Trattando segnali discreti, l'operazione di integrale diventerà una sommatoria, estesa al prodotto di segnale e risposta impulsiva:

$$y(n) = x(n) * h(n) = \sum_{k=-\infty}^{+\infty} x(k)h(n-k)$$

*La sequenza h(n) prende anche il nome di '**kernel' o 'maschera' di convoluzione**'*

# Calcolo convoluzione discreta

- Supponendo limitata la durata della sequenza h(n), il calcolo della convoluzione diventa quindi un semplice algoritmo numerico
  - Si prende il kernel e lo si specularizza, ottenendo h(-k)
  - Si fa scorrere il kernel su tutto il dominio (discreto)
  - Per ogni posizione n si moltiplica punto per punto il segnale d'ingresso x(n) per il kernel traslato h(k-n) e si somma per tutti i k su cui c'è sovrapposizione
  - Il risultato della somma è il valore del segnale d'uscita in n

# Algoritmo (pseudo-codice)

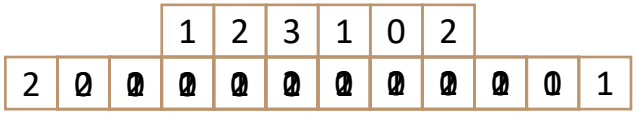
```
int x[N], y[N], h[K]
load x, h
for n in 1...N {
        tmp = 0
        for k in 1...K
                tmp += x[n-K/2+k]*h[k]
        y[n] = tmp
        }
return y
```

*NB. Osserviamo che il calcolo richiede K operazioni di somma/prodotto per campione, quindi la complessità è lineare con la dimensione del kernel → O(K)*

# Esempio di calcolo

$h_{DW}(n)$ | 1 | 0 | 2 | → $h_{DW}(-n)$ | 2 | 0 | 1 |

x(n) | 1 | 2 | 3 | 1 | 0 | 2 |




y(n) | 0 | 1 | 2 | 5 | 5 | 6 | 4 | 0 | 4 | 0 |

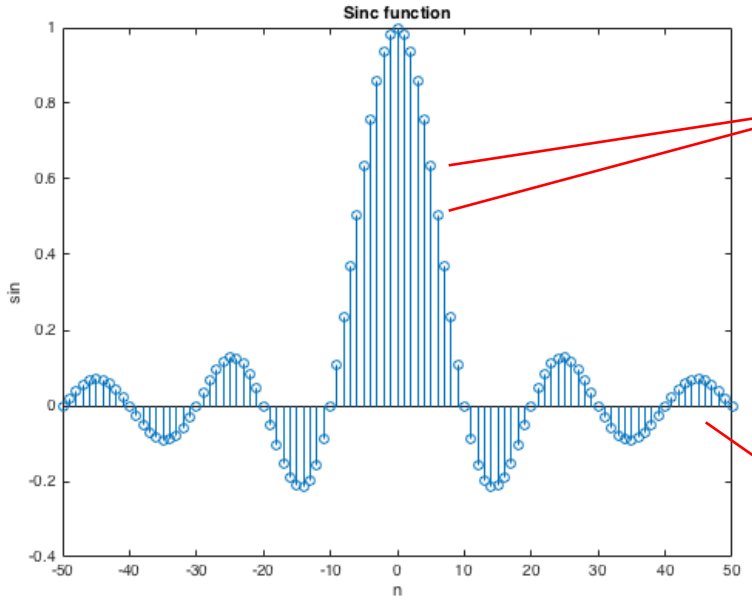
no overlap

# Progetto del filtro

- Ci poniamo ora il problema di come definire il kernel di convoluzione h(k) per ottenere il risultato voluto
  - L'idea è quella di definire il comportamento desiderato in frequenza (es. un filtro passa-basso) e da questo risalire al kernel
  - Occorrerà prima di tutto tornare nel tempo (IFT) e poi campionare la risposta impulsiva ottenuta h(t) per ottenere i coefficienti h(k)
  - **Problema**: partendo da una risposta in frequenza ideale, otterremo un numero di campioni K infinito!
  - Il filtro così definito viene chiamato **IIR** (***infinite impulse response***)

# Esempio: filtro IIR da LPF ideale

- L'LPF ideale ha una risposta in frequenza $H_{LPF}(f)$ rettangolare con banda passante W predefinita
  - La risposta impulsiva corrispondente sarà quindi una sinc
  - Il campionamento dovrà avvenire a frequenza $f_c>2W$ (Nyquist)

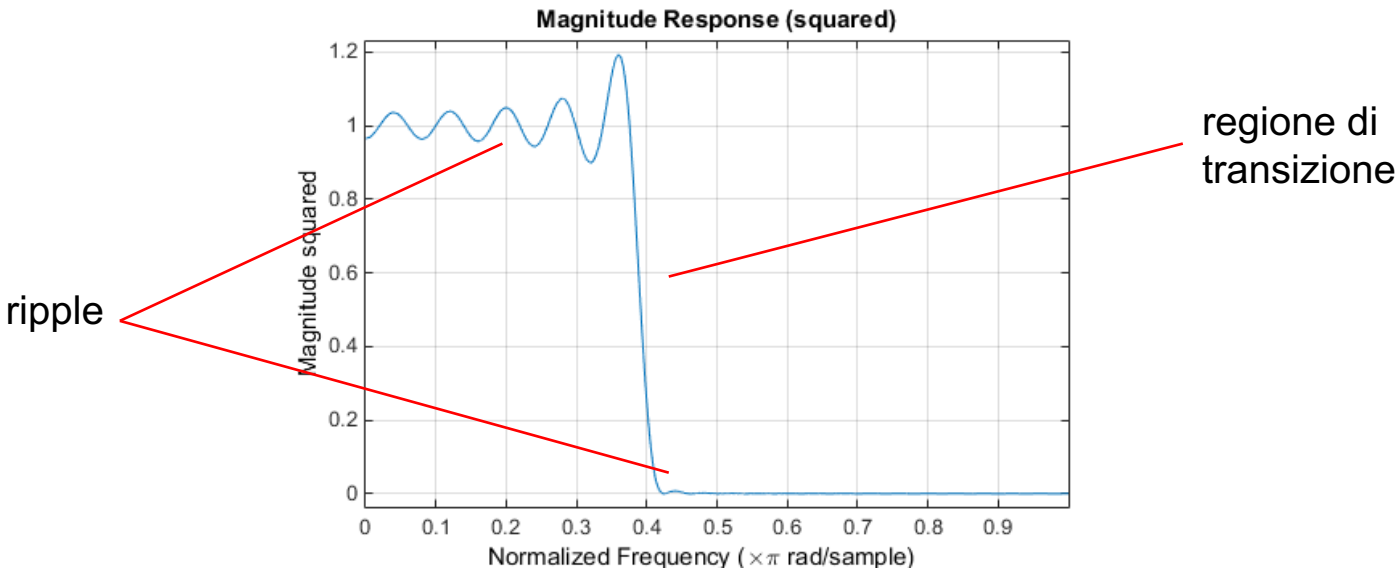


# IIR vs. FIR

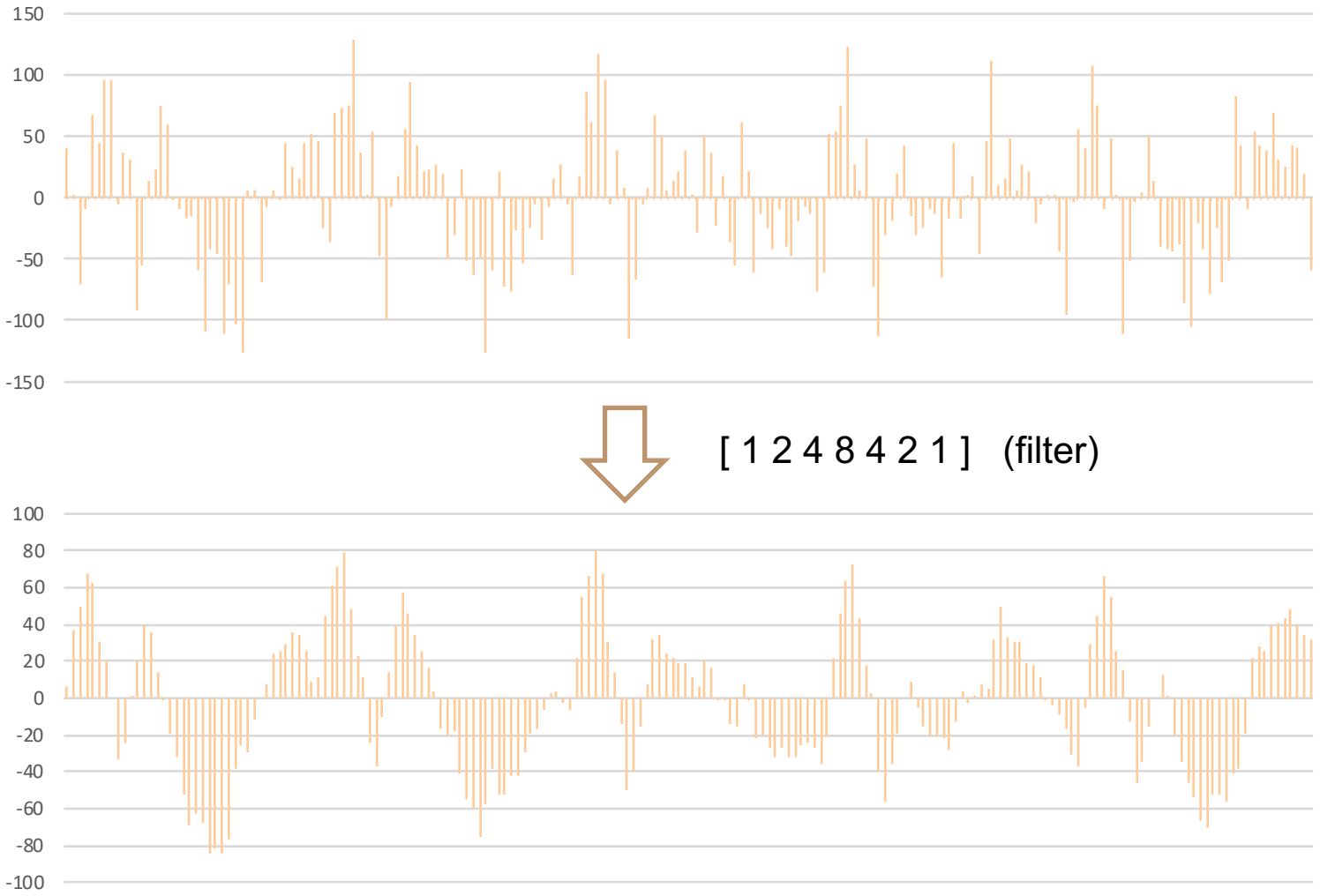
- Per come abbiamo definito la convoluzione, un kernel di dimensione infinita (IIR) richiede infinite somma/prodotti per campione → IRREALIZZABILE TRAMITE CONVOLUZIONE
  - Per renderlo realizzabile dobbiamo tagliare il kernel ad una lunghezza limitata: si può fare **finenstrandolo**
  - Il risultato di questa operazione è il cosiddetto filtro **FIR (finite impulse response)**
  - Notare che con questa operazione introduciamo un'approssimazione (la risposta in frequenza cambia) e un fenomeno di aliasing

# Esempio: filtro FIR da LPF ideale

- Tagliando la risposta impulsiva e tornando in frequenza notiamo che la risposta non è più ideale



# Esempio: FIR LPF su segnale rumoroso



# Esempi comuni di filtri FIR

- Vediamo ora alcuni esempi di kernel che possono essere utilizzati per realizzare semplici filtri FIR
- Ci limitiamo qui a definire alcuni filtri comunemente usati, senza entrare nel dettaglio della progettazione:
  - Filtri passa-basso
    - Media mobile
    - Gaussiano
  - Filtri passa-alto
    - Gradiente
    - Pseudo-Laplaciano

# LPF a media mobile

- Il filtro a media mobile è caratterizzato da un kernel con coefficienti tutti di pari valore:

$$h(k) = \left[ \frac{1}{K} \;\; \frac{1}{K} \;\; \frac{1}{K} \;\; \dots \;\; \frac{1}{K} \right] = \frac{1}{K} \cdot [\, 1 \; 1 \; 1 \; \dots \; 1 \,]$$

  - Pensando all'operazione di convoluzione, si vede facilmente che questo filtro non fa altro che sostituire ogni campione con la media dei campioni che cadono in una finestra di dimensione K centrata su di lui
  - Il nome di media mobile deriva proprio da questo (mobile perché la convoluzione esegue quest'operazione a scorrimento sul segnale)
  - Il fattore di normalizzazione 1/K occorre per evitare che il filtro alteri il valor medio del segnale di ingresso: in generale, la **somma dei coefficienti del kernel in un filtro passa-basso deve essere pari a 1**

# LPF a media mobile: esempio

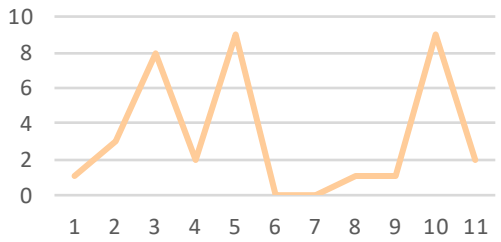
- Prendiamo il segnale $x(n) = [\ 1\ 3\ 8\ 2\ 9\ 0\ 0\ 1\ 1\ 9\ 2\ ]$

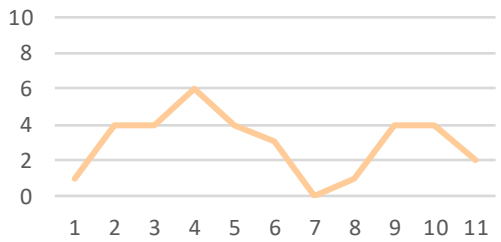
  e un filtro a media mobile $h(k) = [1/3\ \ 1/3\ \ 1/3]\ ,\ K=3$

- Trascurando primo e ultimo campione (per evitare problemi di bordo) avremo:

$$y(n) = [\ 1\ 4\ 4\ 6\ 4\ 3\ 0\ 1\ 4\ 4\ 2\ ]$$

- Il filtro andrà a mediare (e quindi tagliare) le variazioni più intense del segnale, ovvero quelle che cadono nella finestra K

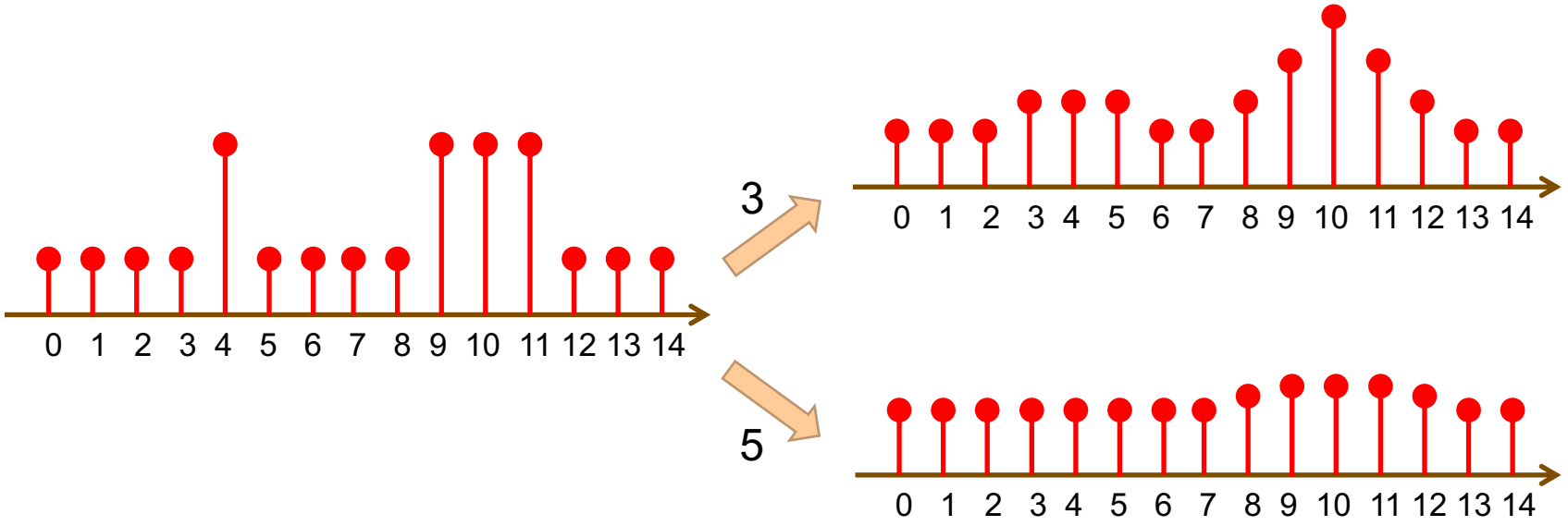



*NB. Il comportamento di un filtro LPF è di tipo integrativo (media pesata)*

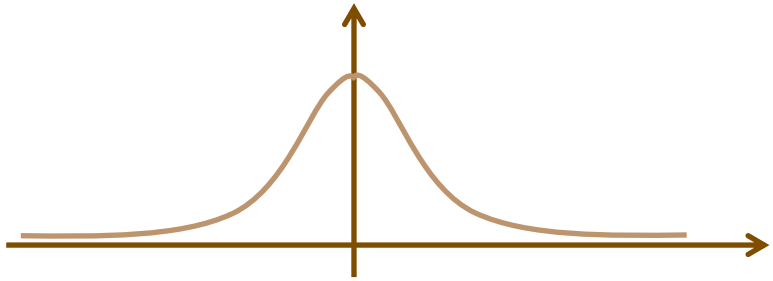
# Intensità del filtro a media mobile

- L'intensità (effetto passa-basso) del filtro sarà, in questo caso, solo proporzionale a K (i coefficienti non possono essere modificati)
- Intuitivamente, più ampia è la finestra del filtro, più estesa è la finestra su cui vado a mediare il segnale di ingresso, tagliando quindi oscillazioni via via più estese (e quindi a minore frequenza)



# LPF Gaussiano

- Per avere un effetto passa-basso più moderato a parità di finestra K, si può agire sui coefficienti
  - L'effetto sarà minore se la media viene pesata, associando valori maggiori al campione centrale e a quelli più vicini
    - Al limite, se il campione centrale viene pesato con un fattore 1 e gli altri con fattore 0, si ottiene un filtro passa-tutto
  - Una classica distribuzione dei coefficienti utilizzata per ottenere questo effetto è quella gaussiana



$$\frac{1}{\sqrt{2\pi\sigma^2}} e^{\frac{(x-\mu)^2}{2\sigma^2}}$$

# LPF gaussiano

- La gaussiana avrà valor medio nullo e varianza $\sigma$
- Più elevato è il valore di $\sigma$, più aumenterà lo spread della gaussiana, producendo una media più estesa e uniforme
  - Al limite per $\sigma$ che tende a zero avremo una funzione impulsiva (coincide con un filtro passa-tutto)
  - A limite per $\sigma$ che tende a infinito avremo una funzione piatta (coincide con un filtro a media mobile)
- Nel caso discreto, i campioni avranno valori:

$$h(k) = \frac{1}{\lambda} e^{-\frac{k^2}{2\sigma^2}} \quad , \qquad k \; in \left[-\frac{K}{2}, \frac{K}{2}\right]$$

dove: $\lambda = \sum_k h(k)$ è il fattore di normalizzazione

$K$ è proporzionale alla deviazione standard (es. K=4$\sigma$)

# LPF gaussiano

- Ad esempio, per $\sigma$=1, avremo K=4 → 5 campioni

$$h(k) = \frac{1}{2.48}\,[0.14 \quad 0.6 \quad 1.0 \quad 0.6 \quad 0.14]$$

- Possiamo fare in modo da ottenere coefficienti interi introducendo un fattore moltiplicativo e approssimando:

$$h(k) = \frac{1}{17}\,[1 \quad 4 \quad 7 \quad 4 \quad 1]$$

# Filtri passa-alto

- Come sappiamo, i filtri passa-alto possono essere ottenuti per differenza a partire dai corrispondenti passa-basso
  - Proviamo ad esempio a realizzare un passa-alto a partire da un filtro passa-basso a media mobile di dimensione 5

$$h_{LPF}(k) = [1/5\ 1/5\ 1/5\ 1/5\ 1/5]$$

$$h_{HPF}(k) = \delta[k] - h_{LPF}(k) = [0\ 0\ 1\ 0\ 0] - [1/5\ 1/5\ 1/5\ 1/5\ 1/5] =$$
$$[-1/5\ -1/5\ 4/5\ -1/5\ -1/5]$$

  - Possiamo semplificare il filtro moltiplicando per 5 (notare che la somma dei coefficienti è pari a zero, quindi non cambia niente

$$h_{HPF}(k) = [-1\ -1\ \ 4\ -1\ -1]$$

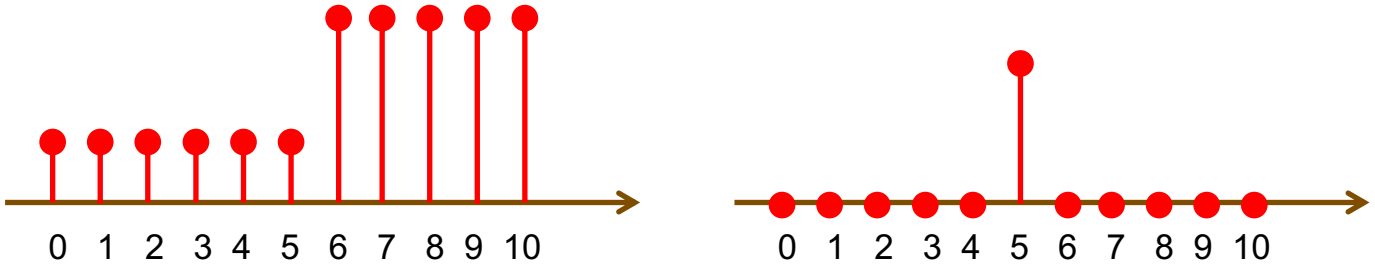
# Fitri passa-alto: osservazioni

- Si nota che il filtro appena definito ha somme 0 (gli LPF avevano somme 1): la componente continua del segnale viene cancellata, come è ovvio per un HPF
  - Non occorre normalizzare!
- Si nota anche un comportamento di tipo differenziale (**segni positivi e negativi**), in pratica l'effetto è derivativo, al contrario degli LPF che erano integrativi
- L'effetto derivativo porta ad evidenziare le variazioni del segnale, attenuandolo quando le variazioni sono lente (nel caso di assenza di variazioni viene addirittura cancellato)

# Filtro gradiente

- Un caso particolarmente semplice di HPF è il filtro gradiente, che esegue una derivata prima discreta del segnale

$$h_{HPF}(k) = [-1\ 1]$$



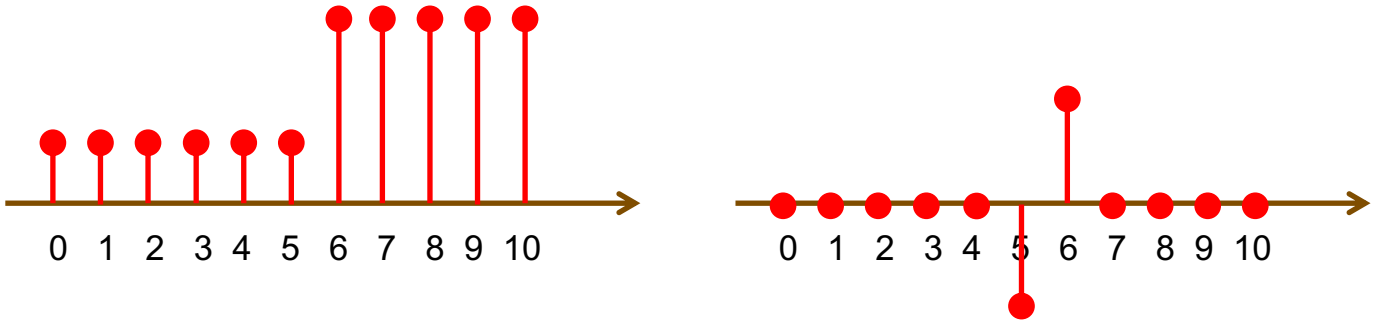

  - Si nota che il filtro evidenzia la transizione a gradino, generando un impulso in corrispondenza del fronte e azzerando le zone piatte

    *NB. Se il gradino fosse stato discendente avremmo avuto un impulso negativo*

# Filtro Laplaciano

- Posso anche eseguire una derivata seconda, tramite il cosiddetto filtro Laplaciano
  - In questo caso avremo una doppia variazione (andamento cosiddetto a 'mexican hat'), ad esempio:

$$h(k) = [0\ -1\ \ 2\ -1\ 0\ ]$$




  - Si nota che in questo caso abbiamo una doppia variazione (doppietto) in corrispondenza della transizione del gradino

# Filtri iterativi

- Dal punto di vista teorico, iterare un filtro ideale non dovrebbe produrre alcun risultato:
  - Prendiamo ad es. un LPF ideale, alla prima applicazione il filtro taglierà le frequenze a partire da un certo valore $f_0$. Applicando nuovamente il filtro andremo a tagliare le stesse frequenze, che però sono già a zero
  - Nella pratica, poiché applichiamo il filtro in maniera approssimata (per ottenere un FIR tagliamo la risposta impulsiva), l'applicazione ripetuta del filtro produrrà ad ogni iterazione un ulteriore taglio
  - Per intensificare l'effetto di un filtro possiamo perciò pensare di iterarlo

# Filtri iterativi: esempio

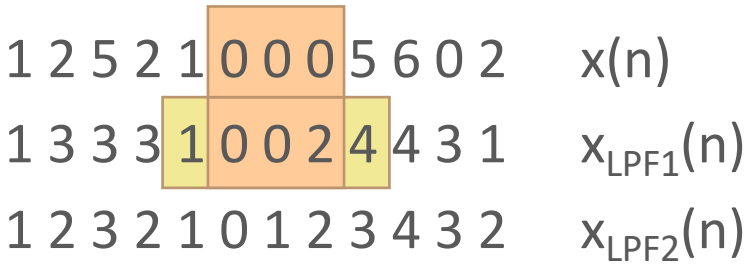
- Consideriamo ad esempio un filtro di media mobile con K=3 e vediamo il risultato di una applicazione iterativa
  - Dato un segnale numerico x(n), vediamo le prime 4 iterazioni

| | |
|---|---|
| 1 2 5 2 1 0 0 0 5 6 0 2 0 1 0 2 3 4 | x(n) |
| 1 3 3 3 1 0 0 2 4 4 3 1 1 0 1 2 3 2 | $x_{LPF1}(n)$ |
| 1 2 3 2 1 0 1 2 3 4 3 2 1 1 1 2 2 2 | $x_{LPF2}(n)$ |
| 1 2 2 2 1 1 1 2 3 3 3 2 1 1 1 2 2 1 | $x_{LPF3}(n)$ |
| 1 2 2 2 1 1 1 2 3 3 3 2 1 1 1 2 2 1 | $x_{LPF4}(n)$ |

  - Si osserva che le prime 3 passate producono variazioni, la quarta no
  - In pratica il filtro 'converge' dopo alcune iterazioni a un risultato stabile

# Filtri iterativi: spiegazione

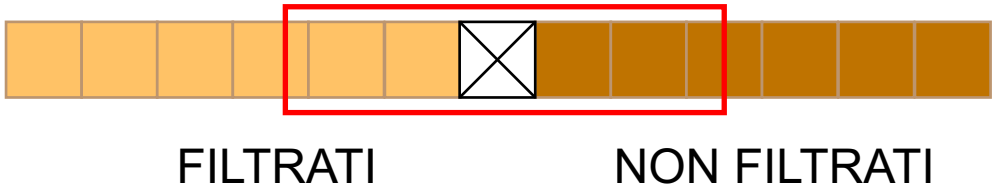
- Poiché dimensione e coefficienti del filtro non cambiano, il motivo del funzionamento iterativo è da cercare altrove
  - Il filtro FIR agisce su un **supporto temporale** limitato dalla dimensione della maschera: alla prima passata metterà in relazione solo campioni a distanza minore di K/2 → non può agire su oscillazioni del segnale (frequenze) con estensione maggiore di K/2
  - L'applicazione iterativa, implicitamente estende il supporto:




*NB. L'effetto progressivamente decresce con la distanza, per cui il filtro converge*

# Cenno ai filtri IIR

- Il problema appena visto non sussisterebbe se potessi fare direttamente dei filtri a supporto infinito (IIR)
  - Come abbiamo detto, questo è impossibile da fare tramite convoluzione perché richiederebbe una complessità infinita
  - Un modo alternativo (approssimato) per realizzare un filtro IIR è tramite meccanismi di **auto-regressione**
  - In pratica, man mano che genero i campioni filtrati li utilizzo per filtrare i campioni successivi (operazione ricorsiva)
  - ATTENZIONE: non è più una convoluzione, anche se ci assomiglia



# Filtri auto-regressivi: esempio

- Vediamo ad esempio l'applicazione di questo concetto ad un filtro di media mobile 3, sulla sequenza vista prima

| | |
|---|---|
| 1 2 5 2 1 0 0 0 5 6 0 2 0 1 0 2 3 4 | x(n) |
| 1 3 3 2 1 0 0 2 4 3 2 1 1 1 2 3 2 1 | $x_{ARMA}(n)$ |
| 1 2 2 2 1 1 1 2 3 3 3 2 1 1 1 2 2 1 | $x_{LPF4}(n)$ |
| 0 1 1 0 0 -1 -1 0 1 0 -1-1 0 0 1 1 0 0 | $X_{ARMA}$-$X_{LPF4}(n)$ |

  - Come si può osservare, l'applicazione del filtro in modalità auto-regressiva genera in un solo passaggio una sequenza molto simile a quella che otteniamo a regime con il filtro FIR iterativo
  - Anche in questo caso il motivo è che estendo virtualmente il supporto, ma in questo modo riesco a farlo in una singola passata (l'effetto del filtraggio viene propagato a tutti i campioni successivi)

# Cenno ai filtri non lineari

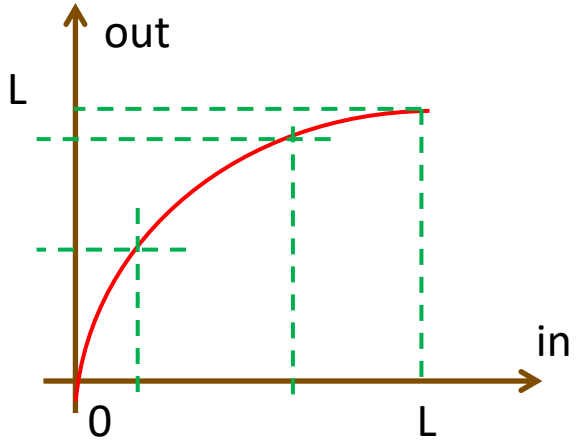
- I filtri non lineari sono molto complessi da realizzare nel dominio analogico (devo usare componentistica non lineare)
- Lavorando su segnali numerici la possibilità di realizzare in modo algoritmico filtri non lineari apre enormi possibilità
- Tipicamente si tratterà di definire opportune relazioni ingresso-uscita, oppure meccanismi più sofisticati di elaborazione
- Vediamo 2 esempi:
  - Rimappatura dei livelli
  - Filtri di rango

# Filtri non lineari: rimappatura livelli

- Abbiamo visto che già l'operazione di quantizzazione è una operazione non lineare
- A valle della quantizzazione, possiamo agire nuovamente in maniera non lineare sui livelli dei campioni, usando funzioni di rimappatura dei livelli
- Posso ad esempio introdurre opportune funzioni per aumentare o ridurre la dinamica del segnale
  - Ad esempio, dato un segnale a L livelli, posso espandere i livelli in un certo intervallo, comprimendo quelli negli altri intervalli

# Rimappatura livelli: esempio

- Prendiamo ad es. la seguente funzione di mappatura (relazione ingresso uscita) non lineare




  - Il segnale filtrato subirà una espansione della dinamica nei livelli più bassi (vicini allo zero) e una compressione ai livelli più alti (vicini a L)

# Rimappatura livelli: risultato

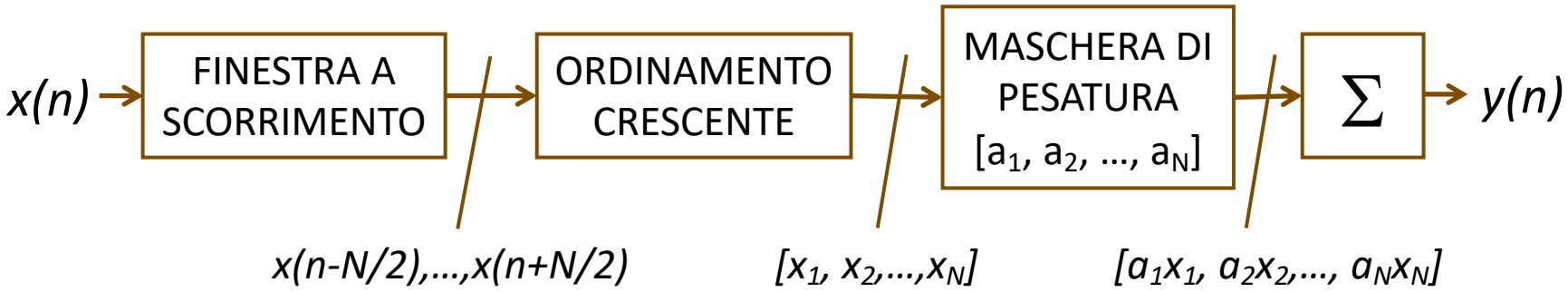
- Operazioni di questo tipo possono essere usate ad es. per:
  - evidenziare (mettere in luce) alcune caratteristiche del segnale, o una parte del suo contenuto
  - Cancellare alcune variazioni
  - Estrarre (per esempio tramite sogliatura) alcune parti azzerandone altre
  - Comprimere la dinamica del segnale (anche in maniera adattativa) per adeguarlo ad un particolare formato o a un dispositivo di rendering

# Filtri non lineari: filtri di rango

- I filtri di rango sono molto popolari nell'elaborazione di segnali (in particolare il filtro mediano), in quanto sono al contempo semplici e potenti
- Il concetto è molto diverso dai filtri FIR, si basano su operazioni di **ordinamento**
  - L'idea è quella di ordinare i valori all'interno della finestra di filtraggio, ed effettuare una media pesata basata sul valore (rango) anziché sulla posizione dei campioni
  - Come per i FIR, l'effetto dipende dalla dimensione della dimensione della maschera e dai pesi utilizzati
  - Poiché l'ordinamento è un'operazione non lineare, non è definita la risposta impulsiva di questo filtro

# Filtri di rango

- Il filtraggio si basa sul seguente schema:




  - Si estrae a scorrimento dal segnale una sequenza di lunghezza N centrata su n
  - Si ordina in senso crescente ottenendo la sequenza ordinata $x_i$, i=1…N
  - Si pesano gli N valori ordinati con altrettanti pesi $a_i$, i=1…N
  - Si sommano i valori pesati $a_i x_i$, i=1…N ottenendo il campione filtrato y(n)

# Filtro di rango: procedura

Sequenza originale

| 1 | 4 | 2 | 1 | 6 | 1 | 2 | 4 | 6 | 1 |
|---|---|---|---|---|---|---|---|---|---|

finestra 3, pesi 1 0 1



1 4 2 → 1 4 2 1 6 1 2 4 6 1 → 1 4 2 1 6 1 2 4 6 1 → 1 4 2 1 6 1 2 4 6 1 → 1 4 2 1 6 1 2 4 6 1 → 1 4 2 1 6 1 2 4 6 1 → 1 4 2 1 6 1 2 4 6 1 → 1 4 2 1 6 1 2 4 6 1 → 1 4 2 1 6 1 2 4 6 1 = 7

Sequenza filtrata (senza bordi)

| 5 | 5 | 7 | 7 | 7 | 5 | 8 | 7 |
|---|---|---|---|---|---|---|---|

# Filtri di rango: pseudo-codice

```
int x[N], y[N], a[K]
int temp, buff[K]
load x, a
for n in K/2...N-K/2 {
    tmp = 0
    buff = sort(x+n-K/2, K)
    // ordina K elem di x da n-K/2 a n+K/2 e li mette in buff
    for k=1...K
       tmp += buff[k]*a[k]
    y[n] = tmp
    }
return y
```

# Filtri di rango

- Il risultato dipende chiaramente dai pesi $a_i$
- Alcuni semplici filtri di rango sono i filtri di massimo, minimo e mediano
  - MASSIMO: $a_i = \begin{cases} 0 & \forall i \neq N \\ 1 & i = N \end{cases}$
  - MINIMO: $a_i = \begin{cases} 0 & \forall i \neq 1 \\ 1 & i = 1 \end{cases}$
  - MEDIANO: $a_i = \begin{cases} 0 & \forall i \neq \left\lfloor \frac{N}{2} \right\rfloor \\ 1 & i = \left\lfloor \frac{N}{2} \right\rfloor \end{cases}$

# Filtri di rango: massimo e minimo

- I filtri di massimo e minimo sostituiscono il valore centrale della finestra con i valori massimo/minimo nella finestra
  - In pratica, vado ad eliminare picchi indesiderati nel segnale (*outlaiers*)
  - ESEMPIO (filtro di minimo, eliminazione outlier verso l'alto):

x(n)=[1 2 2 1 1 0 9 1 2 1 0 1]

↓

filtro di minimo, N=3

↓

y(n)=[1 1 1 1 0 0 0 1 1 0 0 0]

*NB.* *- Tutta la sequenza viene scalata su valori più bassi*

*- Non si introducono nuovi valori nella sequenza filtrata*

# Filtri di rango: mediano

- Nel mediano seleziono il valore centrale anziché gli estremi
  - Riesco ad eliminare contemporaneamente outlier in alto e in basso
  - ESEMPIO:

x(n)=[5 4 4 5 3 4 9 4 5 0 5 4 5]

↓

filtro mediano, N=3

↓

y(n)=[5 4 4 4 4 4 4 5 4 5 4 5 5]

*NB:* *- Anche in questo caso non introduco valori nuovi*

*- In questo caso, non ho scalamento della sequenza verso alto/basso*

# Mediano: utilizzo

- Il mediano è normalmente usato come filtro di 'regolarizzazione' del segnale
  - Si eliminano variazioni non volute, in particolare di tipo **impulsivo**
  - In presenza di picchi isolati di rumore, riesco a eliminarli senza introdurre effetto passa-basso
  - ESEMPIO: confronto mediano-media mobile

$$x(n) = [4\ 4\ 4\ 4\ 9\ 4\ 4\ 4\ 4\ 0\ 4\ 4\ 4\ 4]$$

$$y_{MA}(n) = [4\ 4\ 4\ 6\ 6\ 6\ 4\ 4\ 3\ 3\ 3\ 4\ 4\ 4]$$

$$y_{MED}(n) = [4\ 4\ 4\ 4\ 4\ 4\ 4\ 4\ 4\ 4\ 4\ 4\ 4\ 4]$$

  - Il filtro LPF abbassa e 'allarga' (*spreading*) i picchi di rumore ma non riesce a rimuoverli, il filtro mediano li elimina completamente

# Mediano: regolazione potenza del filtro

- Regolando l'ampiezza della finestra posso modulare l'effetto del filtro
  - Chiaramente, a finestre più ampie corrisponderà un effetto più pesante
  - Posso anche misurare in maniera esatta l'effetto del filtro, in termini di sequenze su cui opera. Si dimostra che:
  - **Un filtro mediano di larghezza *N* lascia inalterate sequenze monotone (anche non strettamente) di lunghezza maggiore di $\left\lfloor \frac{N}{2} \right\rfloor + 1$**
  - ESEMPIO

$$x(n)=[0\ 0\ 0\ 0\ 0\ 0\ 1\ 0\ 0\ 0\ 0\ 1\ 1\ 0\ 0\ 0\ 0\ 1\ 1\ 1\ 1\ 1\ 1]$$

$$y_{MED3}(n) = [0\ 0\ 0\ 0\ 0\ 0\ 0\ 0\ 0\ 0\ 0\ 1\ 1\ 0\ 0\ 0\ 1\ 1\ 1\ 1\ 1\ 1\ 1]$$

$$y_{MED5}(n) = [0\ 0\ 0\ 0\ 0\ 0\ 0\ 0\ 0\ 0\ 0\ 0\ 0\ 0\ 0\ 0\ 0\ 1\ 1\ 1\ 1\ 1\ 1]$$

# Filtri di rango: complessità

- La complessità dei filtri di rango è maggiore di un FIR
  - Infatti, facciamo esattamente le stesse operazioni, a cui si aggiunge l'operazione di ordinamento
  - Usando algoritmi semplici (es. bubblesort), l'ordinamento ha a sua volta complessità quadratica, in questo caso avremo $O(2K^2)$ operazioni per campione (K = finestra del filtro)
  - Usando algoritmi più sofisticati (es. quicksort) l'ordinamento scende a complessità O(NlogN), ma questo conviene solo per filtri molto grandi
  - Per filtri particolarmente semplici (massimo, minimo, mediano) non occorre in generale rifare l'ordinamento del vettore ad ogni posizione, ma si può semplicemente sostituire il valore entrante a quello uscente